

Neural Word Representation

MLP LAB
KyungTae Lim



Contents

01 Word Representation

02 What the word embedding is learning?

03 Word2Vec

04 Applying Word Embedding

05 Deep Contextualized Word Embeddings

06 ELMo and BERT



01. Word Representation

Word Representation

- A representation method of symbolic characters as computable numbers

Word Representation

- A representation method of symbolic characters as computable numbers
- Can we just represent the word as numbers? Consider the word "tree" like humans think

• signifier (symbol) $\Leftrightarrow$ signified (idea or thing)

= denotational semantics

tree $\Leftrightarrow$ {  ,  ,  , ... }

1

:

N

- When we think of a concept of a word, we also imply the semantics of the word, right?

Word Representation(one-hot)

- Does One-hot represent the semantics of the word?

- Word Dictionary $V = \{\text{cat, fat, mat, sat, the, on}\}$

- $\text{cat} = [1, 0, 0, 0, 0, 0]$

- $\text{fat} = [0, 1, 0, 0, 0, 0]$

- $\text{mat} = [0, 0, 1, 0, 0, 0]$

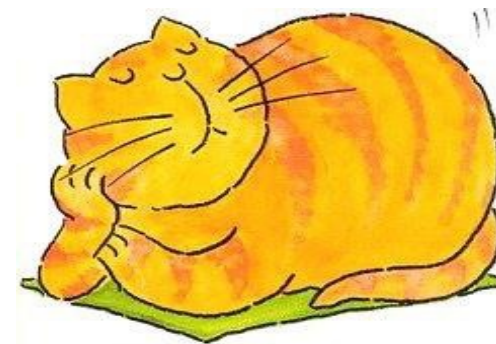
- $\text{sat} = [0, 0, 0, 1, 0, 0]$

- $\text{the} = [0, 0, 0, 0, 1, 0]$

- $\text{on} = [0, 0, 0, 0, 0, 1]$

Human: "The fat cat sat
on the mat"

Computer: "32 832 561
634 6132 32 565"



?!?!?

- $|V|$ is very large, requiring a very large dimensional sparse vector
- Does it contain the **meaning (semantics)** of the word?

Problem with words as discrete symbols (one-hot)

- One-hot does not represent the meaning of a word with a localist representation

Example: in web search, if a user searches for “Seattle motel”, we would like to match documents containing “Seattle hotel”

But:

motel = [0 0 0 0 0 0 0 0 0 0 1 0 0 0 0]

hotel = [0 0 0 0 0 0 0 1 0 0 0 0 0 0 0]

These two vectors are orthogonal

There is no natural notion of **similarity** for one-hot vectors!

Solution:

- Could try to rely on WordNet’s list of synonyms to get similarity?
 - But it is well-known to fail badly: incompleteness, etc.
- **Instead: learn to encode similarity in the vectors themselves**

Solution

- Dimensionality: Reduce dimensions by changing a sparse representation to a dense one
- Semantic encoding: Similar words are represented with similar vector values.

Example: in web search, if a user searches for “Seattle motel”, we would like to match documents containing “Seattle hotel”

motel = [0 0 0 0 0 0 0 0 0 0 1 0 0 0 0]

hotel = [0 0 0 0 0 0 0 1 0 0 0 0 0 0 0]

These two vectors are orthogonal

There is no natural notion of **similarity** for one-hot vectors!

Solution:

motel = [0.34 -1.22 3.31]

hotel = [0.30 -1.01 3.31]

now “motel” and “hotel” is similar

Semantic Encoding: let similar words be represented by similar vector values



How we do that?

Representing words by their context

Representing words by their context



- **Distributional semantics:** A word's meaning is given by the words that frequently appear close-by
 - *"You shall know a word by the company it keeps"* (J. R. Firth 1957: 11)[1]
 - One of the most successful ideas of modern statistical NLP!
- When a word w appears in a text, its **context** is the set of words that appear nearby (within a fixed-size window).
- We use the many contexts of w to build up a representation of w

...government debt problems turning into	banking	crises as happened in 2009...
...saying that Europe needs unified	banking	regulation to replace the hodgepodge...
...India has just given its	banking	system a shot in the arm...

These **context words** will represent **banking**

The goal of this lecture

We will build a dense vector for each word, chosen so that it is similar to vectors of words that appear in similar contexts, measuring similarity as the vector **dot** (scalar) **product**

$$\begin{array}{l} \textit{banking} = \begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \end{pmatrix} \end{array} \qquad \begin{array}{l} \textit{monetary} \\ \text{화폐의} \end{array} = \begin{pmatrix} 0.413 \\ 0.582 \\ -0.007 \\ 0.247 \\ 0.216 \\ -0.718 \\ 0.147 \\ 0.051 \end{pmatrix}$$

Note: **word vectors** are also called **(word) embeddings** or **(neural) word representations** They are a **distributed** representation

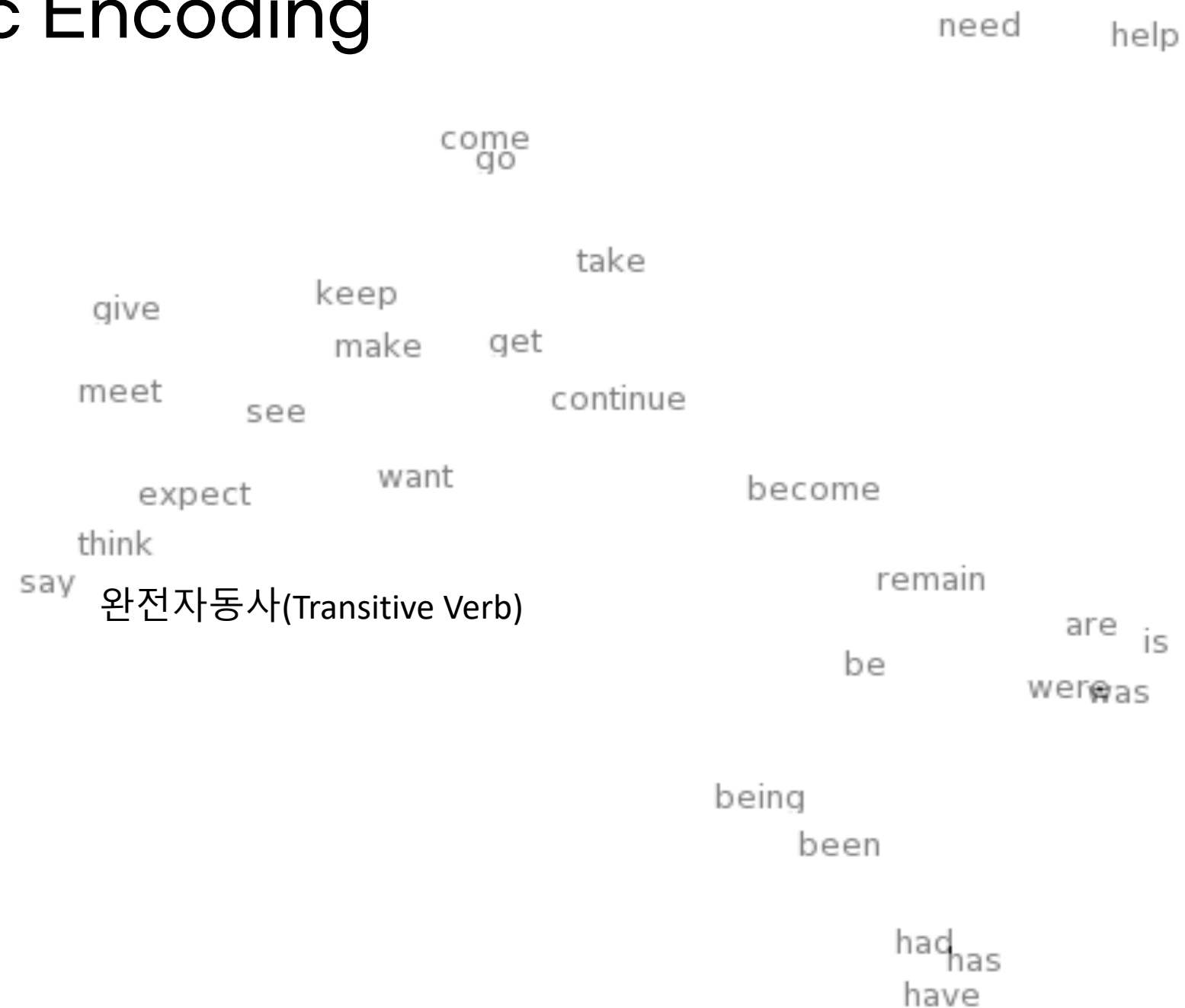
Dot product: $0.118+0.462+0.001-0.026+0.024+0.390+0.051+0.014 \approx 1.03$

Word meaning as a neural word vector - visualization

- Visualization of Semantic Encoding

expect =

0.286
0.792
-0.177
-0.107
0.109
-0.542
0.349
0.271
0.487



One-hot VS. Word embedding

	One-hot	Word Embedding
Dimensionality	High-dimensional	Low-dimensional
Representation type	Sparse	Dense
Representation method	Manual calculations	Learning from training data
Type of value	0 or 1	Real number

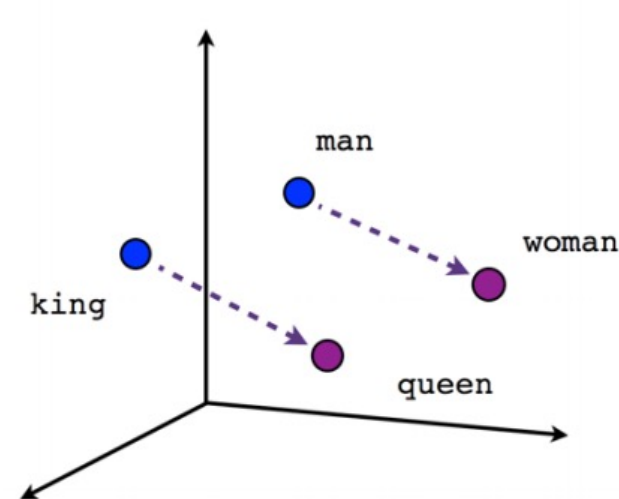
02. What the word embedding is learning?

But does word embedding really represent
the **meaning of the word**?

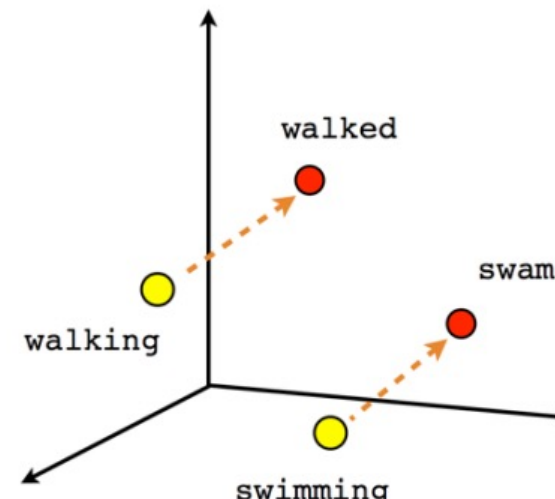


Investigation of Word Embedding

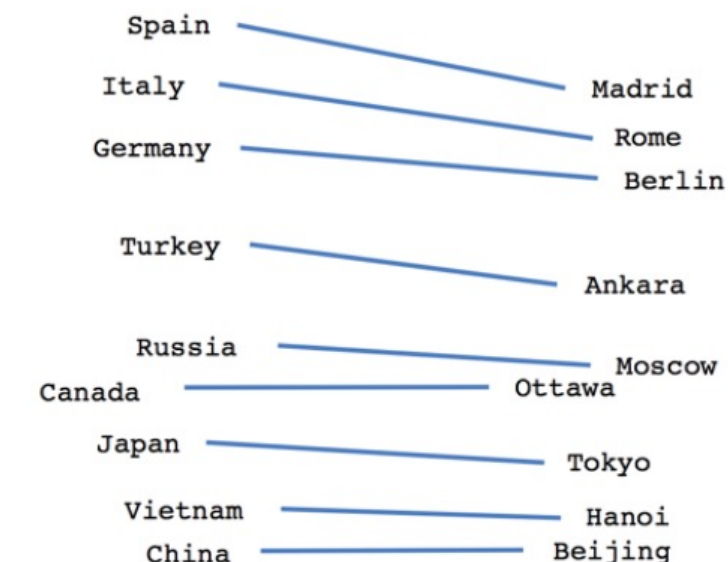
- Semantic Encoding: let **similar words** be represented by **similar vector values**
= **‘Words that occur in similar places have similar meanings’** (Distributional Hypothesis)



Male-Female



Verb tense



Country-Capital

한국-서울+도쿄

QUERY

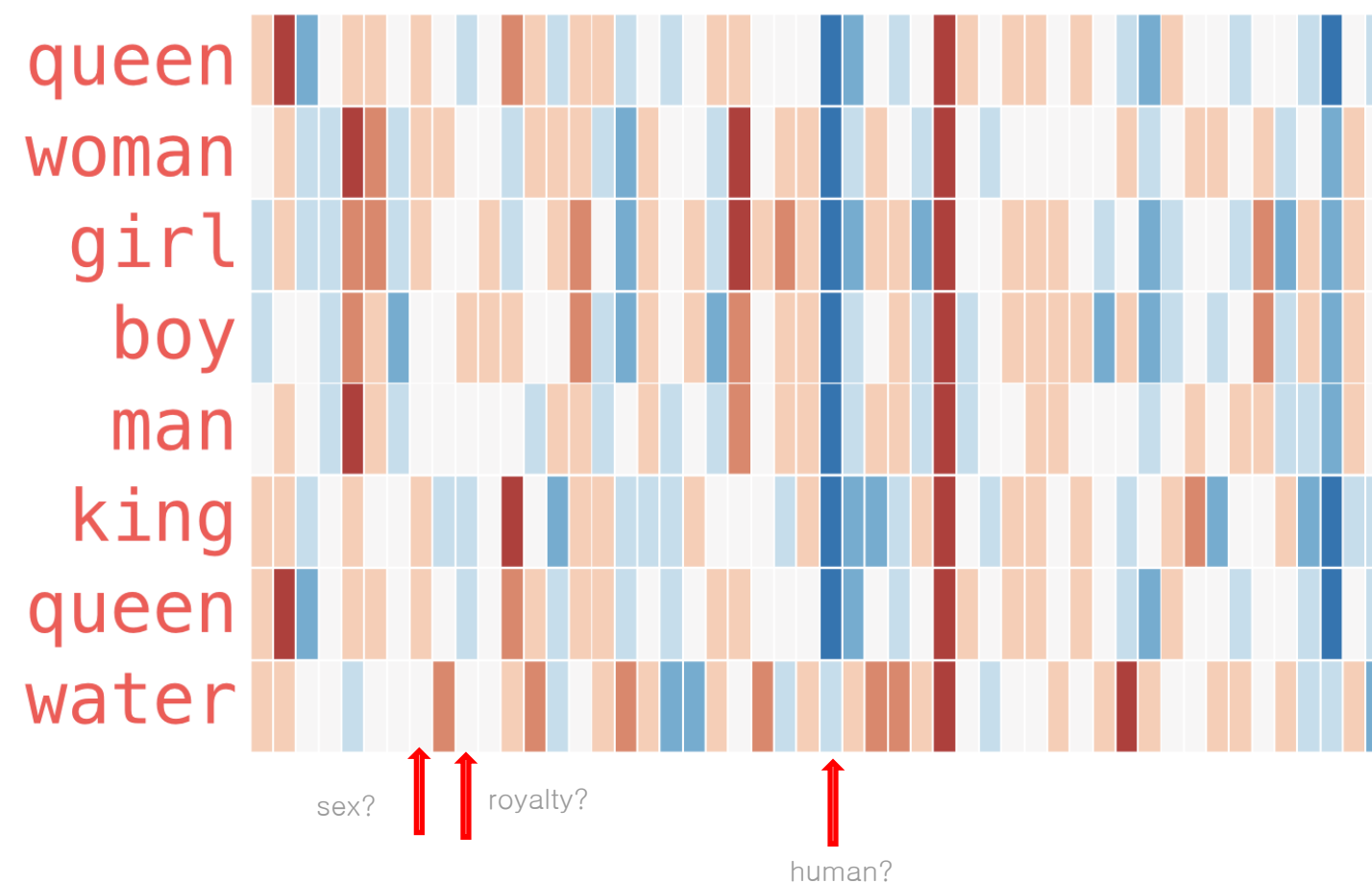
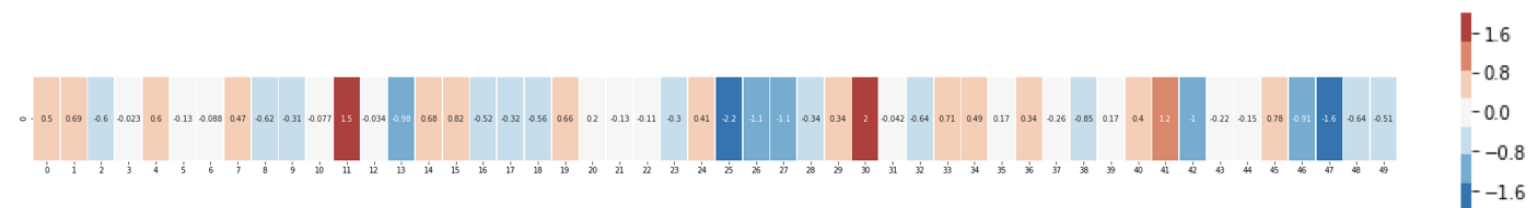
+한국/Noun +도쿄/Noun -서울/Noun

RESULT

일본/Noun

Investigation of Word Embedding

- What does the value of the number from trained word embeddings indicate?
 - It's not clear what features each dimension represents, but you can make guesses by comparisons.



[Observation Point].
Unclear what each column implies
Similarity between "man" and "boy" and "woman" and "girl"
Columns that imply human when compared to "water"
There is a column that can be assumed to be a royalty

[0.50451 , 0.68607 , -0.59517 , -0.022801, 0.60046 , -0.13498 , -0.08813 , 0.47377 , -0.61798 , -0.31012 ,
-0.076666, 1.493 , -0.034189, -0.98173 , 0.68229 , 0.81722 , -0.51874 , -0.31503 , -0.55809 , 0.66421 , 0.1961
, -0.13495 , -0.11476 , -0.30344 , 0.41177 , -2.223 , -1.0756 , -1.0783 , -0.34354 , 0.33505 , 1.9927 ,
-0.04234 , -0.64319 , 0.71125 , 0.49159 , 0.16754 , 0.34344 , -0.25663 , -0.8523 , 0.1661 , 0.40102 , 1.1685 ,
-1.0137 , -0.21585 , -0.15155 , 0.78321 , -0.91241 , -1.6106 , -0.64426 , -0.51042]

**Now we just need to know the algorithm to
train word embedding, right?**



03. Word2Vec

Word2Vec Overview

- Word2vec (Mikolov et al. 2013) is a framework for training word vectors

Idea:

- We have a large corpus (“body”) of text: a long list of words
- Every word in a fixed vocabulary is represented by a **vector**
- Go through each position t in the text, which has a center word c and context (“outside”) words o
- Use the **similarity of the word vectors** for c and o to **calculate the probability** of o given c (or vice versa)
- **Keep adjusting the word vectors** to maximize this probability



Word2Vec Overview

- Word2vec (Mikolov et al. 2013) is a framework for training word vectors
- There are two different type of Word2Vec

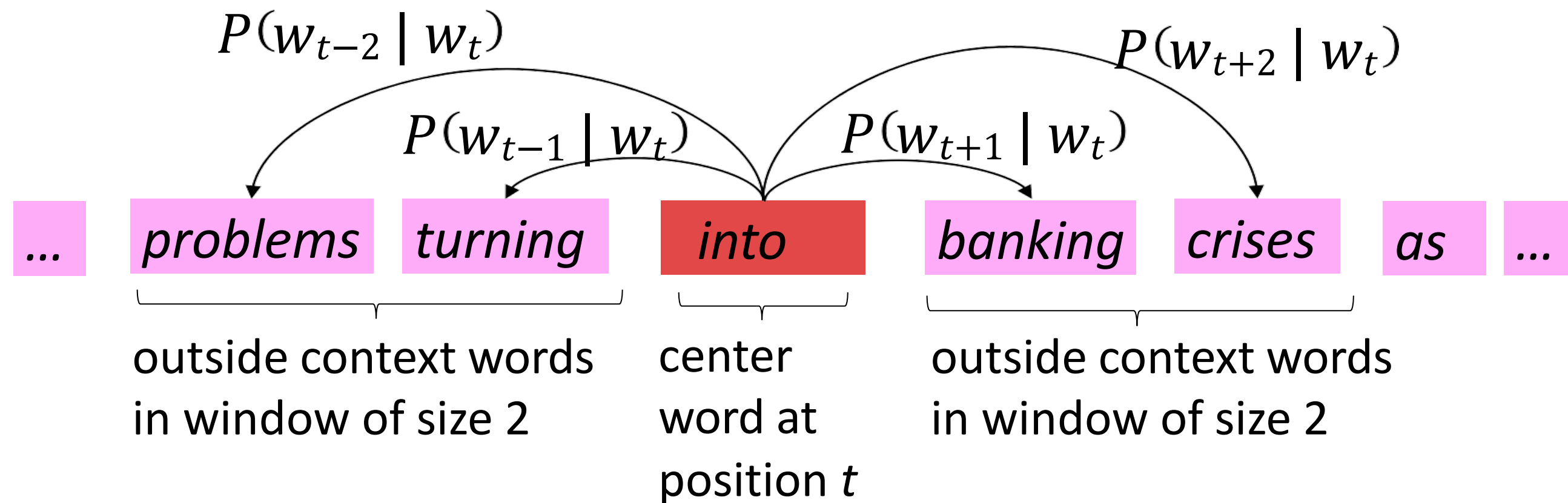
Skipgram: A method to predict surrounding words from a word in the middle

CBOW(Continuous Bag of Words): A method to predict a center word based on given words around the center one.

Word2Vec Overview (Skipgram)

- Word2vec (Mikolov et al. 2013) is a framework for training word vectors

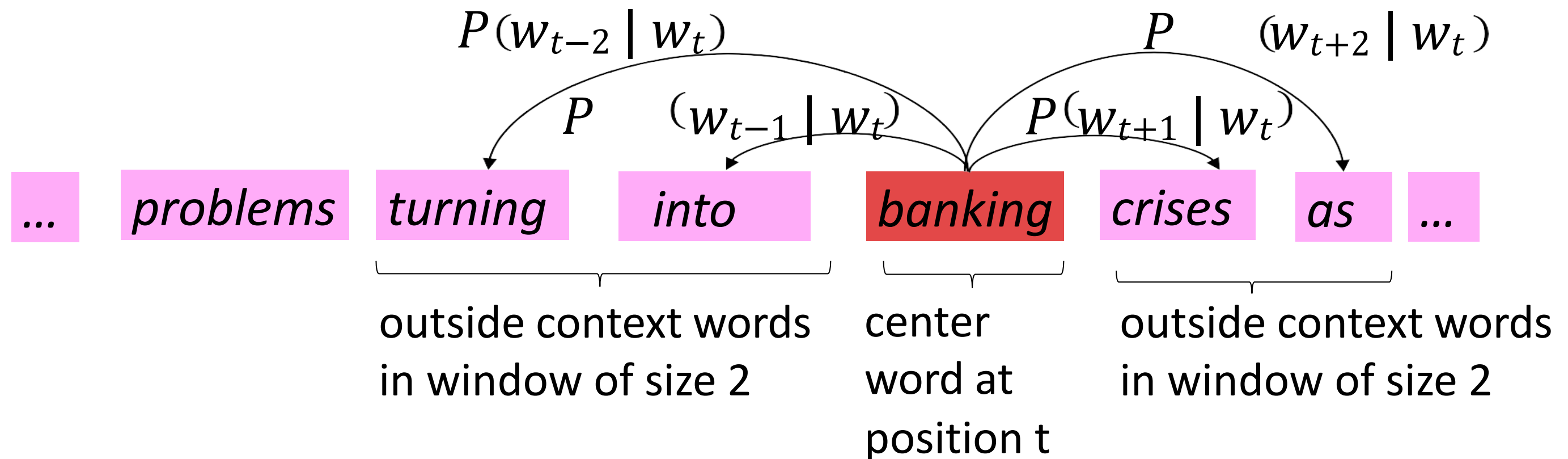
Example windows and process for computing $P(w_{t+j} | w_t)$



Word2Vec Overview (Skipgram)

- Sliding Window with the window size 2

Example windows and process for computing $P(w_{t+j} | w_t)$



Word2Vec Overview (Skipgram)

- An example of training Data with the window size 2

window size=2. see the two words left and right.

Center word

Context words

The fat cat sat on the mat

The fat cat sat on the mat

The fat cat sat on the mat

The fat cat sat on the mat

The fat cat sat on the mat

The fat cat sat on the mat

The fat cat sat on the mat

Center word	Context words
[1, 0, 0, 0, 0, 0, 0]	[0, 1, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0]
[0, 1, 0, 0, 0, 0, 0]	[1, 0, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0]
[0, 0, 1, 0, 0, 0, 0]	[1, 0, 0, 0, 0, 0, 0], [0, 1, 0, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0]
[0, 0, 0, 1, 0, 0, 0]	[0, 1, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 1, 0, 0]	[0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 0, 1, 0], [0, 0, 0, 0, 0, 0, 1]
[0, 0, 0, 0, 0, 1, 0]	[0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 0, 0, 1]
[0, 0, 0, 0, 0, 0, 1]	[0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 0, 1, 0]

Word2Vec Objective Function

- Word2vec is trained based on Negative Log Likelihood (NLL)

For each position $t = 1, \dots, T$, predict context words within a window of fixed size m , given center word w_t . Data likelihood:

Likelihood = $L(\theta) = \prod_{t=1}^T \prod_{\substack{-m \leq j \leq m \\ j \neq 0}} P(w_{t+j} | w_t; \theta)$ The fat cat sat on the mat

θ is all variables to be optimized

$\left[\begin{array}{c} [0, 0, 0, 1, 0, 0, 0] \\ [0, 1, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0], \\ [0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 0, 1, 0] \end{array} \right]$

sometimes called a *cost* or *loss* function

The **objective function** $J(\theta)$ is the (average) negative log likelihood:

$$J(\theta) = -\frac{1}{T} \log L(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log P(w_{t+j} | w_t; \theta)$$

Minimizing objective function $\Leftrightarrow$ Maximizing predictive accuracy

Word2Vec Objective Function

- Let's minimize our objective function!

- We want to minimize the objective function:

$$J(\theta) = -\frac{1}{T} \log L(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log P(w_{t+j} \mid w_t; \theta)$$

- **Question:** How to calculate $P(w_{t+j} \mid w_t; \theta)$?
- **Answer:** We will *use two* vectors per word w :
 - v_w when w is a center word
 - u_w when w is a context word
- Then for a center word c and a context word o :

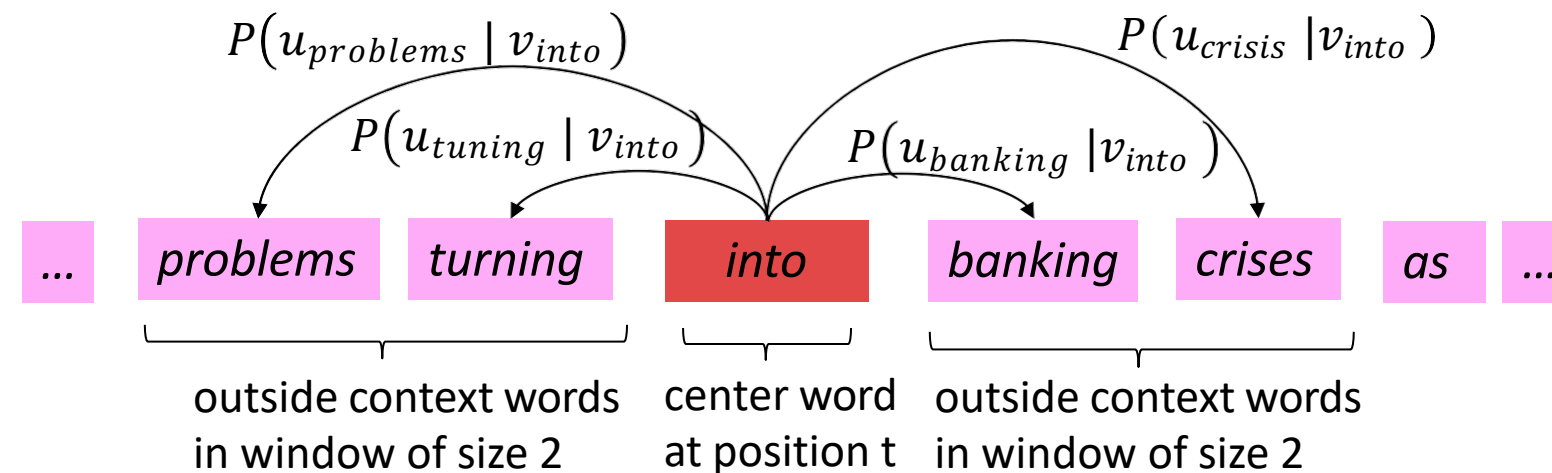
$$P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$$

Word2Vec Objective Function

- Let's minimize our objective function!

- Example windows and process for computing $P(w_{t+j} | w_t)$
- $P(u_{problems} | v_{into})$ short for $P(problems | into ; u_{problems}, v_{into}, \theta)$

All words vectors θ
appear in denominator



Word2Vec Objective Function

- Goal: let **similar words** be represented by **similar vector values**
 - Loss: Among all the words, how proportionally (or relatively) similar are the given center word and outside word?

② Exponentiation makes anything positive

$$P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$$

① Dot product compares similarity of o and c .

$$u^T v = u \cdot v = \sum_{i=1}^n u_i v_i$$

Larger dot product = larger probability

③ Normalize over entire vocabulary to give probability distribution

- This is an example of the **softmax function** $\mathbb{R}^n \rightarrow (0,1)^n$

$$\text{softmax}(x_i) = \frac{\exp(x_i)}{\sum_{j=1}^n \exp(x_j)} = p_i$$

Open region

- The softmax function maps arbitrary values x_i to a probability distribution p_i
 - “max” because amplifies probability of largest x_i
 - “soft” because still assigns some probability to smaller x_i
 - Frequently used in Deep Learning

But sort of a weird name because it returns a distribution!

Word2Vec Objective Function

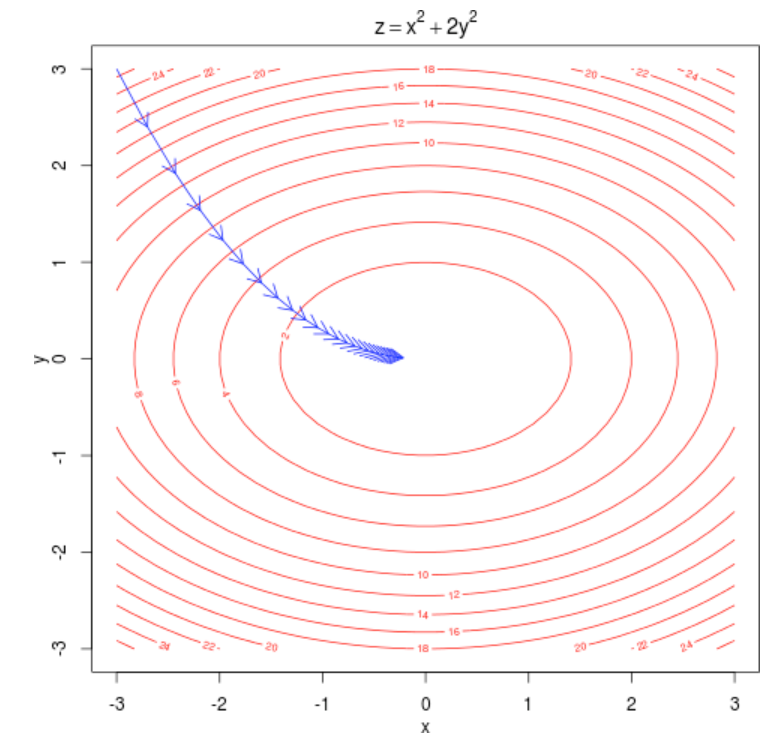
- Let's optimize our objective function!

To train a model, we gradually adjust parameters to minimize a loss

- Recall: θ represents **all** the model parameter in one long vector
- In our case, with d -dimensional vectors and V -many words, we have
- Remember: every word has two vectors



$$\theta = \begin{bmatrix} v_{aardvark} \\ v_a \\ \vdots \\ v_{zebra} \\ u_{aardvark} \\ u_a \\ \vdots \\ u_{zebra} \end{bmatrix} \in \mathbb{R}^{2dV}$$



- We optimize these parameters by walking down the gradient (see right figure)
- We compute **all** vector gradients!

Word2Vec Structure (the Skipgram model)

- Let's Train the Skipgram word embedding with Window size 3 of the first center word

(a) Create an one-hot encoding w_t by extracting all used words from document D.

(b) Randomly initialize the word embedding parameter W (W in Figure, Center word vector) that we will eventually train,

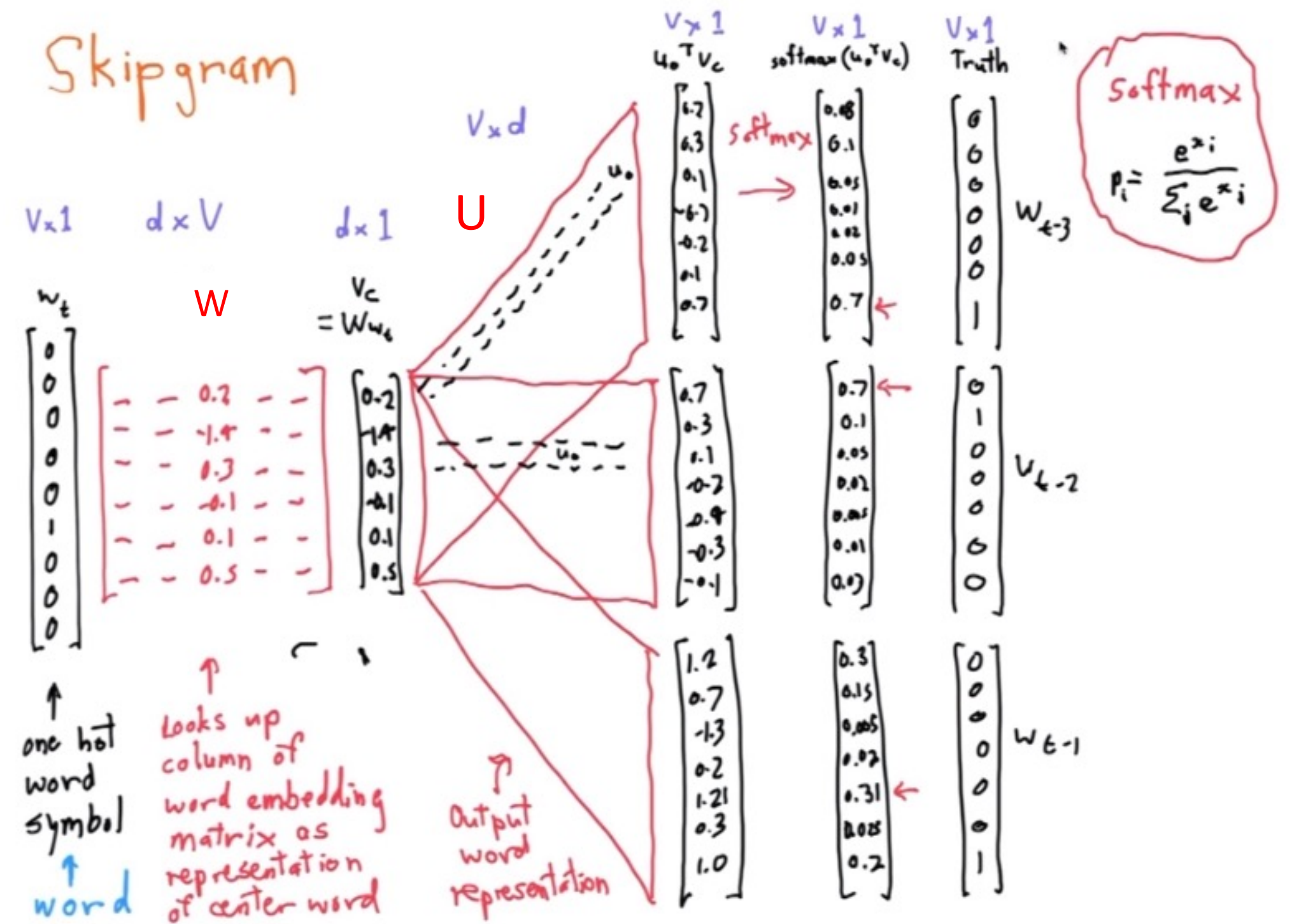
(c) Randomly initialize the scoring parameter U (three red squares, Context word vector).

(d) Forward operations: $w_t * W = v_c$ (word embedding),
 $v_c * U = u_c$ (word embedding * scoring parameter),
 $\text{softmax}(u_c) = y'$ (* is the vector product)

(e) Loss function: Cross-Entropy

(f) Backward: Gradient Decent Computation

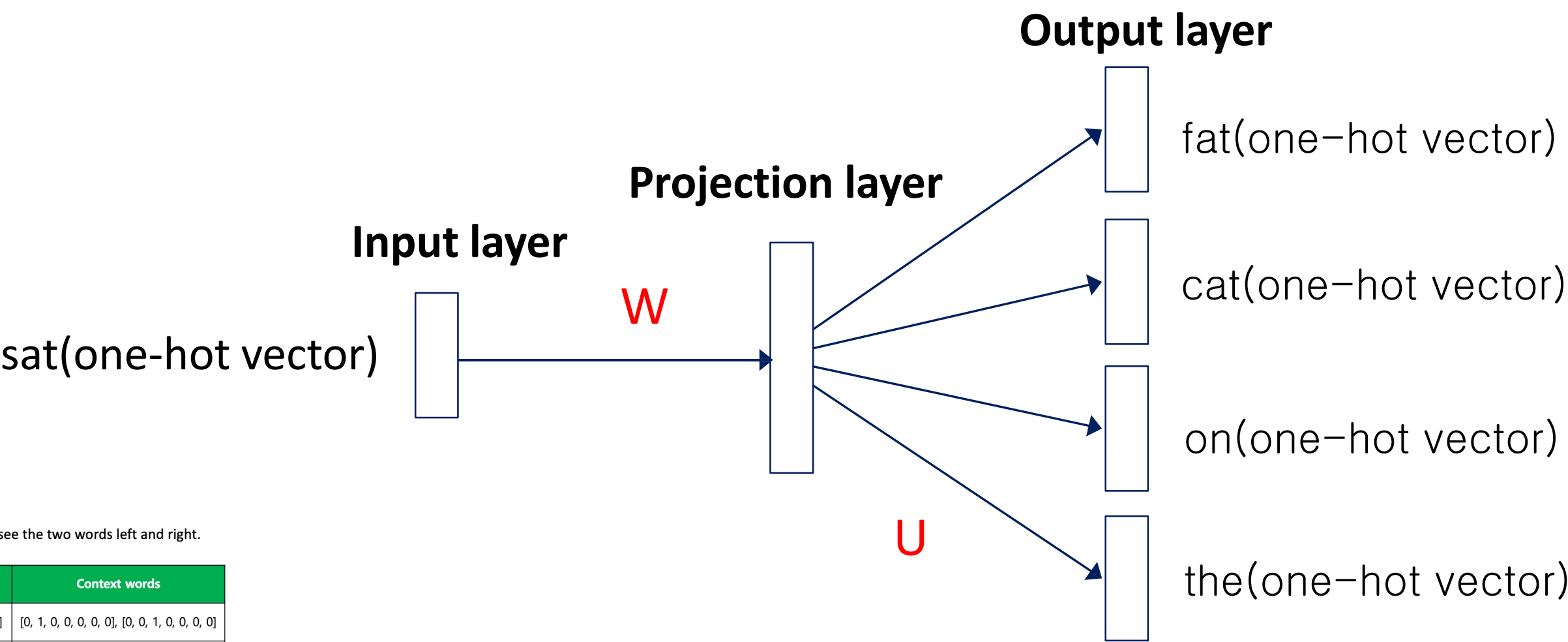
(g) Update Parameters: Update parameters W, U based on the results of Gradient Decent.



Word2Vec Structure (the Skipgram model)

- A simplified structure of the Skipgram

<https://blog.naver.com/jujbob/221147997064>



Center word Context words

↓ ↓

The fat cat sat on the mat

The fat cat sat on the mat

The fat cat sat on the mat

The fat cat sat on the mat

The fat cat sat on the mat

The fat cat sat on the mat

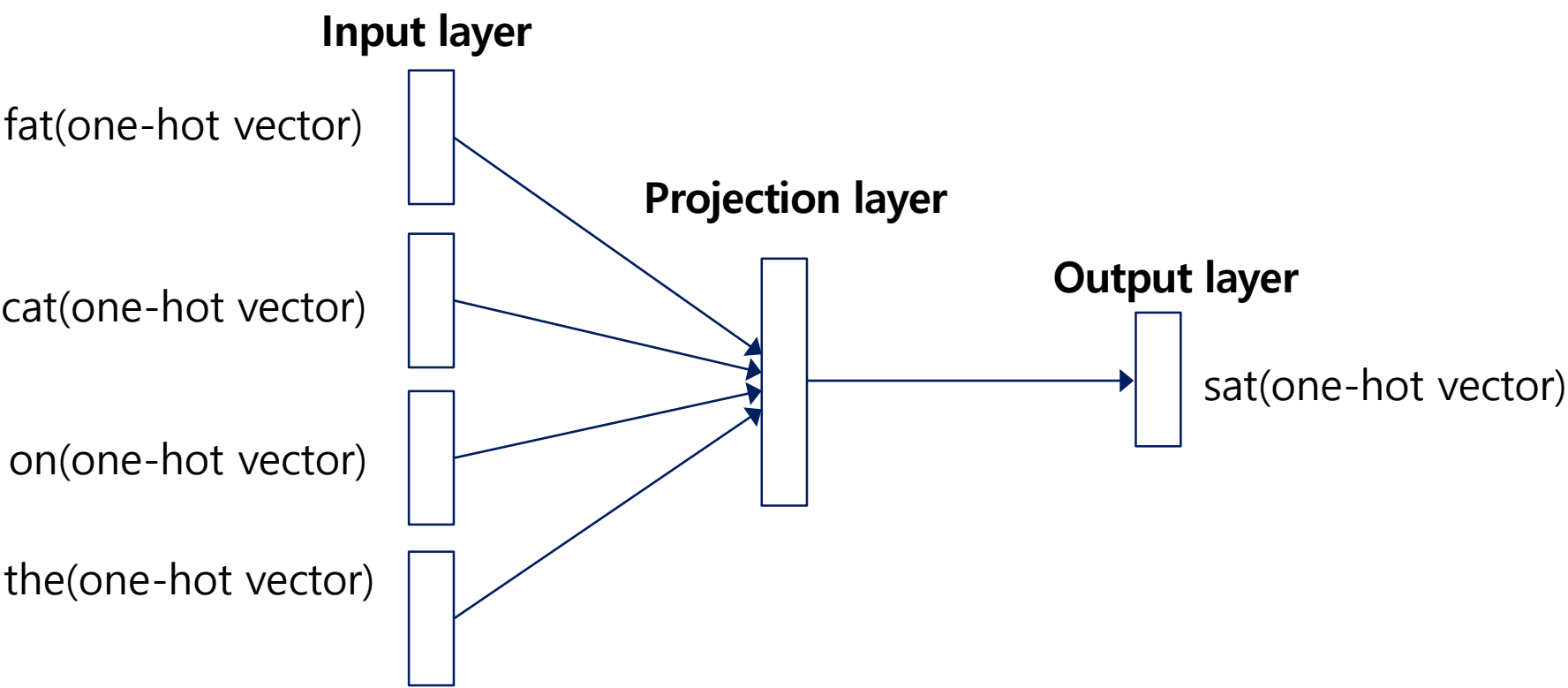
The fat cat sat on the mat

window size=2. see the two words left and right.

Center word	Context words
[1, 0, 0, 0, 0, 0, 0]	[0, 1, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0]
[0, 1, 0, 0, 0, 0, 0]	[1, 0, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0]
[0, 0, 1, 0, 0, 0, 0]	[1, 0, 0, 0, 0, 0, 0], [0, 1, 0, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0]
[0, 0, 0, 1, 0, 0, 0]	[0, 1, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 1, 0, 0]	[0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 0, 1, 0], [0, 0, 0, 0, 0, 0, 1]
[0, 0, 0, 0, 0, 1, 0]	[0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 0, 0, 1]
[0, 0, 0, 0, 0, 0, 1]	[0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 0, 1, 0]

Word2Vec Structure (the CBOW model)

- A simplified structure of the CBOW



window size=2. see the two words left and right.

Center word Context words

↓ ↓

The fat cat sat on the mat

The fat cat sat on the mat

The fat cat sat on the mat

The fat cat sat on the mat

The fat cat sat on the mat

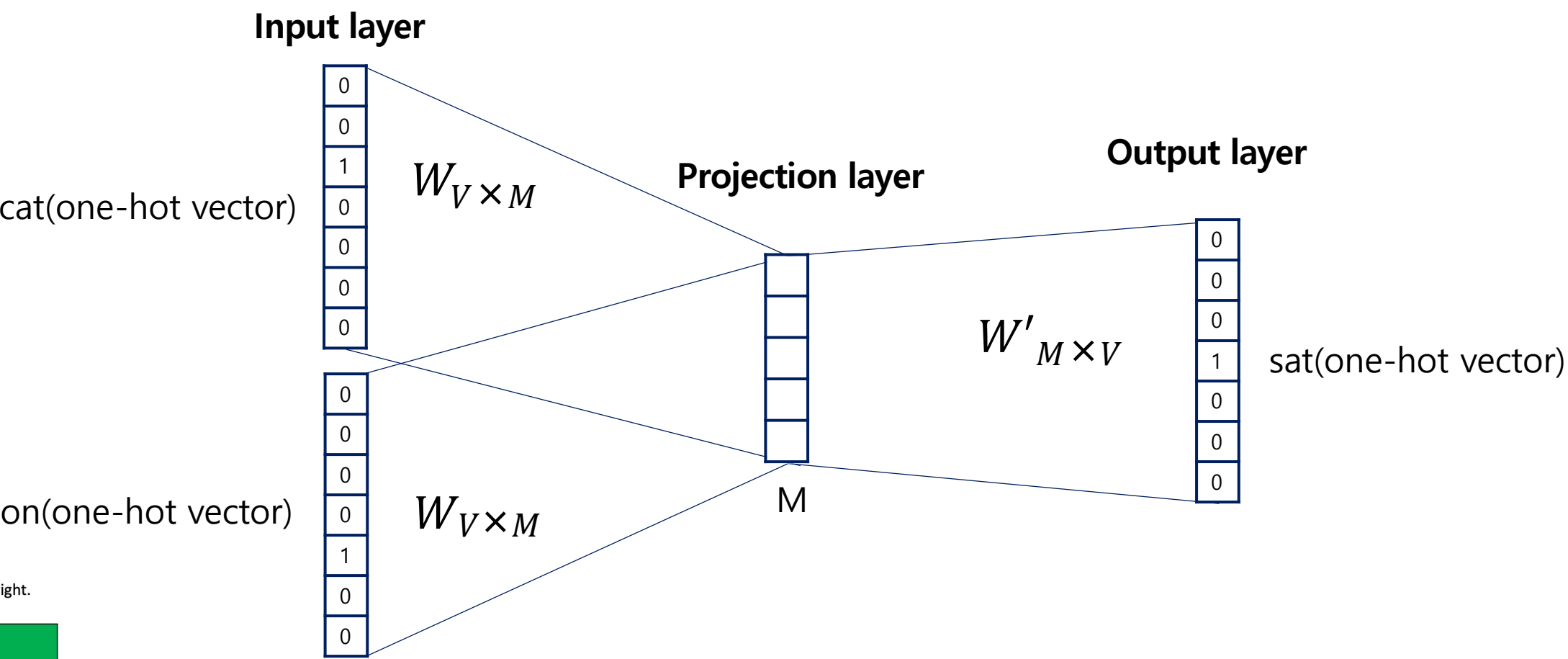
The fat cat sat on the mat

The fat cat sat on the mat

Center word	Context words
[1, 0, 0, 0, 0, 0, 0]	[0, 1, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0]
[0, 1, 0, 0, 0, 0, 0]	[1, 0, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0]
[0, 0, 1, 0, 0, 0, 0]	[1, 0, 0, 0, 0, 0, 0], [0, 1, 0, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0]
[0, 0, 0, 1, 0, 0, 0]	[0, 1, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0]
[0, 0, 0, 0, 1, 0, 0]	[0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 0, 1, 0]	[0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 0, 1, 0], [0, 0, 0, 0, 0, 0, 1]
[0, 0, 0, 0, 0, 0, 1]	[0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 0, 1, 0]

Word2Vec Structure (the CBOW model)

- A simplified structure of the CBOW



Center word
↓
The fat cat sat on the mat

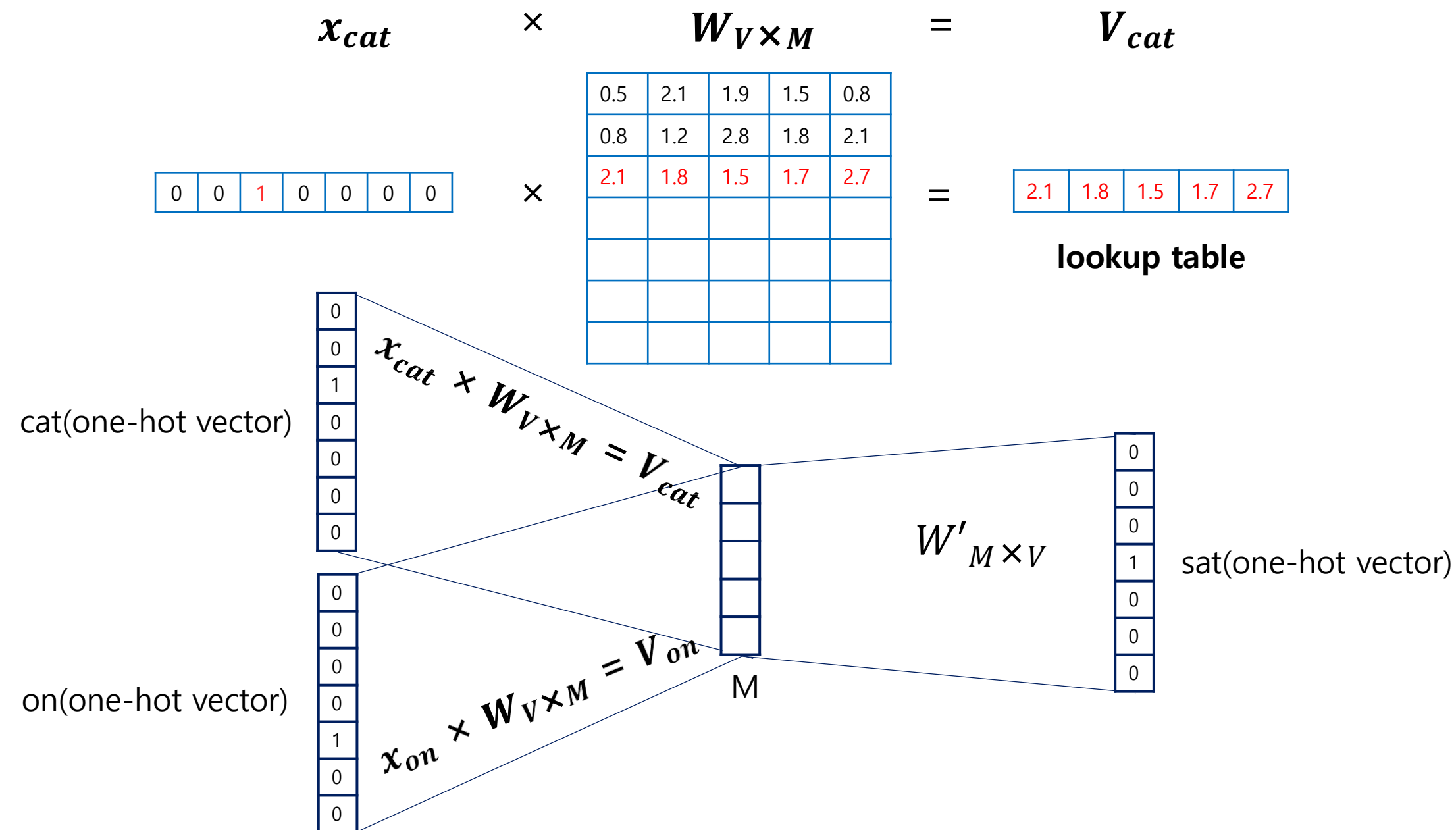
Context words
↙ ↘
The fat cat sat on the mat
The fat cat sat on the mat
The fat cat sat on the mat
The fat cat sat on the mat
The fat cat sat on the mat
The fat cat sat on the mat
The fat cat sat on the mat

window size=2. see the two words left and right.

Center word	Context words
[1, 0, 0, 0, 0, 0, 0]	[0, 1, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0]
[0, 1, 0, 0, 0, 0, 0]	[1, 0, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0]
[0, 0, 1, 0, 0, 0, 0]	[1, 0, 0, 0, 0, 0, 0], [0, 1, 0, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0]
[0, 0, 0, 1, 0, 0, 0]	[0, 1, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 1, 0, 0]	[0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 0, 1, 0], [0, 0, 0, 0, 0, 0, 1]
[0, 0, 0, 0, 0, 1, 0]	[0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 0, 0, 1]
[0, 0, 0, 0, 0, 0, 1]	[0, 0, 0, 0, 0, 1, 0], [0, 0, 0, 0, 0, 0, 1]

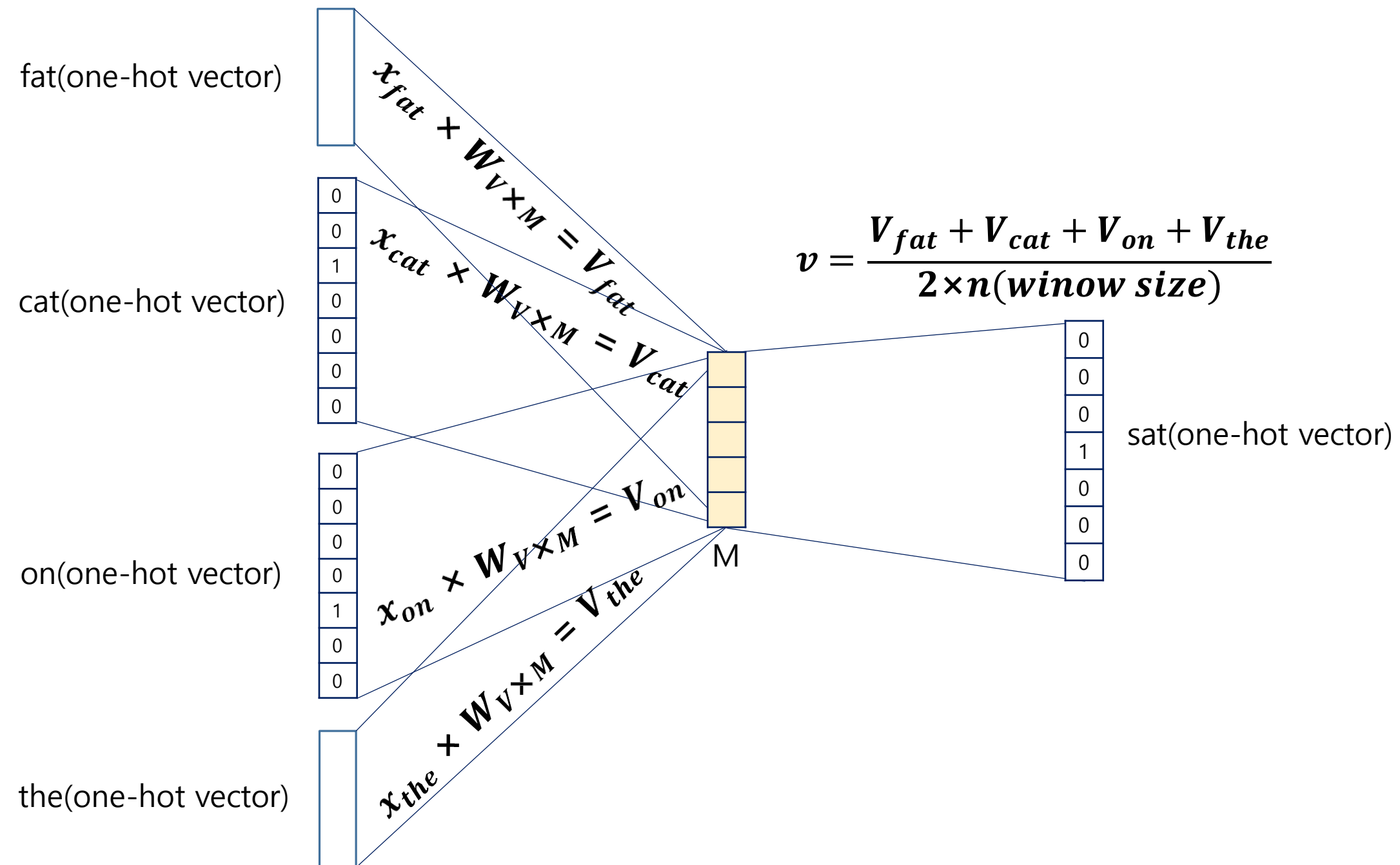
Word2Vec Structure (the CBOW model)

- A simplified structure of the CBOW



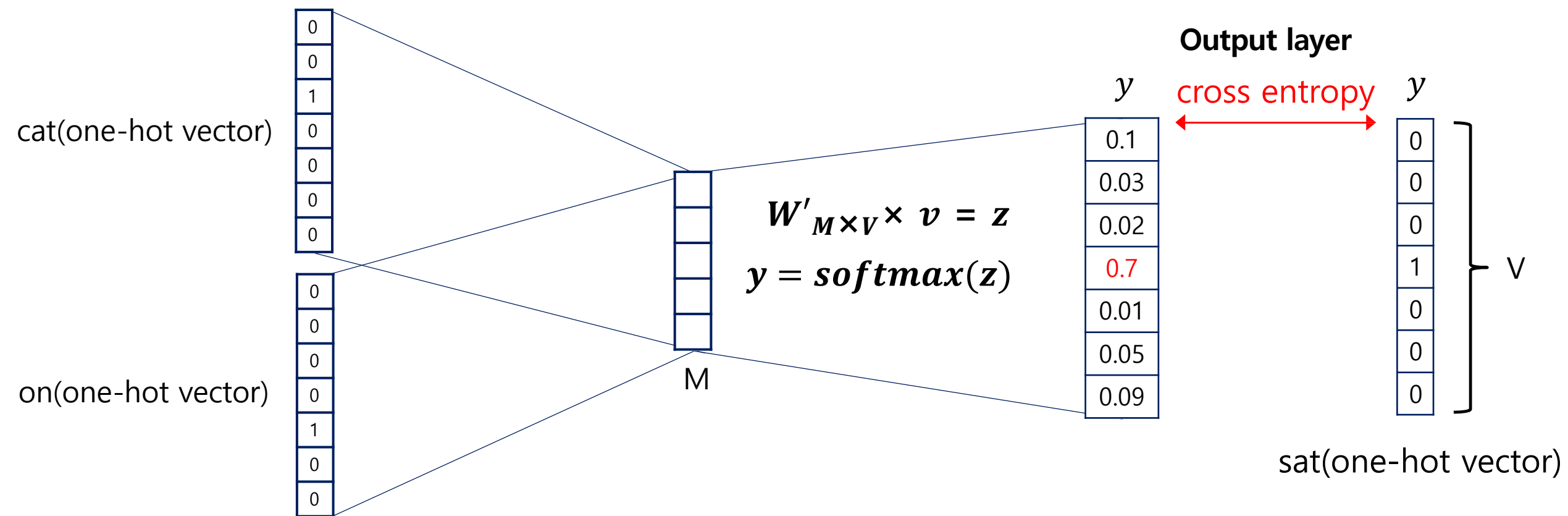
Word2Vec Structure (the CBOW model)

- A simplified structure of the CBOW



Word2Vec Structure (the CBOW model)

- A simplified structure of the CBOW



04. Applying Word Embeddings

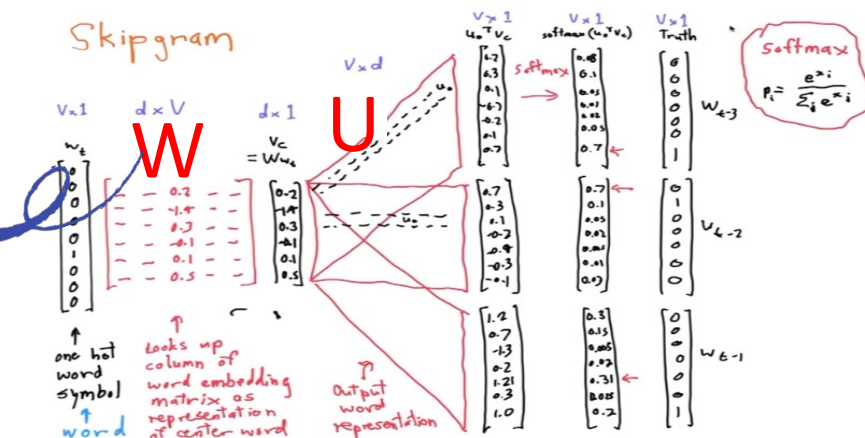
How to use word embeddings?

- (1) Pretraining on plain texts and then save parameter **W** as a file
- (2) Load the saved file and use the vector as like tfidf!

(2) Load “a.emb” for another NLP system

Dimension (D)

(1) save
a.emb



	음주운전	사고	가해자	강력	처벌	역주행	사건	집행유예	면허	취소	규정
음주운전	1	0	0	0	0	0	0	0	0	0	0

One-Hot Encoding Vector

	음주운전	사고	가해자	강력	처벌	역주행	사건	집행유예	면허	취소	규정
음주운전	0.23	0.43	0.32	0.67	0.29	0.56	0.45	0.31	...	...	...
사고	...	...	...	...	...	...	...	...	...	...	...
가해자	...	...	...	...	...	...	...	...	...	...	...
강력	...	...	...	...	...	...	...	...	...	...	...
처벌	...	...	...	...	...	...	...	...	...	...	...
역주행	...	...	...	...	...	...	...	...	...	...	...
사건	...	...	...	...	...	...	...	...	...	...	...
집행유예	...	...	...	...	...	...	...	...	...	...	...
면허	...	...	...	...	...	...	...	...	...	...	...
취소	...	...	...	...	...	...	...	...	...	...	...
규정	...	...	...	...	...	...	...	...	...	...	...

Weight Matrix

	음주운전	사고	가해자	강력	처벌	역주행	사건	집행유예	면허	취소	규정
음주운전	0.23	0.43	0.32	0.67	0.29	0.56	0.45	0.31	...	...	...

Word Embedding Vector

How to train Word2Vec?

- We can use well-coded libraries like Genism or Fasttex.. etc

```
1 from gensim.models import Word2Vec
2
3 embedding_model = Word2Vec(df_drop['token_final'],
4                             sg = 1,
5                             size = 1000,
6                             window = 2,
7                             min_count = 1,
8                             workers = 4)
9
10 print(embedding_model)
11
12 model_result = embedding_model.wv.most_similar("음주운전")
13 print(model_result)
```

Word2Vec(vocab=30036, size=1000, alpha=0.025)

[('살인죄', 0.8733304738998413), ('형량', 0.8702058792114258), ('상해', 0.8652387261390686), ('음주', 0.8635096549987793), ('고의', 0.8585751056671143), ('의료사', 0.8552800416946411), ('벌금형', 0.8549568057060242), ('살인', 0.8530592918395996), ('강간', 0.852802038192749), ('전과', 0.849102258682251)]

How to save the trained Word2Vec?

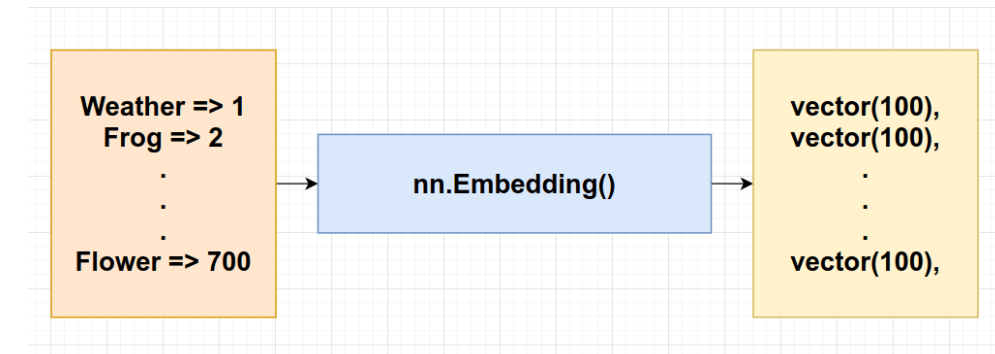
- We can use well-coded libraries like Gensim or FastText.. etc

```
1 from gensim.models import KeyedVectors
2
3 embedding_model.wv.save_word2vec_format
4 ('/content/drive/MyDrive/GitHub/MachineLearning/PythorchDeepLearningProject/국민청원 분류하기/petitions_tokens_w2v')
5 loaded_model = KeyedVectors.load_word2vec_format
6 ('/content/drive/MyDrive/GitHub/MachineLearning/PythorchDeepLearningProject/국민청원 분류하기/petitions_tokens_w2v')
7
8 model_result = loaded_model.most_similar("음주운전")
9 print(model_result)
```

```
[('살인죄', 0.8733304738998413), ('형량', 0.8702058792114258), ('상해',
0.8652387261390686), ('음주', 0.8635096549987793), ('고의',
0.8585751056671143), ('의료사', 0.8552800416946411), ('벌금형',
0.8549568057060242), ('살인', 0.8530592918395996), ('강간',
0.852802038192749), ('전과', 0.849102258682251)]
```


How to load the trained Word2Vec on Pytorch??

- Pytorch provides the nn.Embedding class.
 - Use as nn.Embedding(# of words, dimensionality of words)
 - emb = nn.Embedding(10, 3) randomly generates a 3-dimensional word vector for 10 words.
 - emb([0, 2]) returns vectors for the 0th word (3-dimensional vector), the 2nd word



```
>>> # an Embedding module containing 10 tensors of size 3
>>> embedding = nn.Embedding(10, 3)
>>> # a batch of 2 samples of 4 indices each
>>> input = torch.LongTensor([[1,2,4,5],[4,3,2,9]])
>>> embedding(input)
tensor([[[[-0.0251, -1.6902,  0.7172],
          [-0.6431,  0.0748,  0.6969],
          [ 1.4970,  1.3448, -0.9685],
          [-0.3677, -2.7265, -0.1685]],
        [[ 1.4970,  1.3448, -0.9685],
          [ 0.4362, -0.4004,  0.9400],
          [-0.6431,  0.0748,  0.6969],
          [ 0.9124, -2.3616,  1.1151]]]])
```

$$x_{cat} \times W_{V \times M} = V_{cat}$$

0	0	1	0	0	0	0
---	---	---	---	---	---	---

LongTensor[[2]]

×

0.5	2.1	1.9	1.5	0.8
0.8	1.2	2.8	1.8	2.1
2.1	1.8	1.5	1.7	2.7

embedding

=

2.1	1.8	1.5	1.7	2.7
-----	-----	-----	-----	-----

output

How to load the trained Word2Vec on Pytorch??

- Let's do it yourself
 - Train Word2vec with Naver movie review data (or whatever you want) and save the embedding as a file
 - Read the saved file and initialize it by putting it into a randomly initialized nn.Embedding
 - Investigate and Train your won word embedding with Fasttex, Skipgram, CBOW.. So far
 - Compare each trained word embedding with different algorithms using word similarity
 - ~~- use your won embedding into NLI or NER problems (Team assignment for two weeks later)~~
 - ~~— 1. you need a person who trains word embedding~~
 - ~~— 2. you need a person who build a NER or NLI system~~
 - ~~— 3. you need a person who can present overall your system~~

05. Deep Contextualized Word Embedding

We will discuss it later ...

06. ELMO, BERT

We will discuss it later ...

Thank you